

CONSULTAS AVANÇADAS EM SQL

Dominando Filtros, Ordenação e Agregação com Python

Duração:	120 minutos
Nível:	Intermediário
Tecnologias:	Python, SQLite, SQL
Pré-requisitos:	Conhecimento básico de SQL e Python

SUMÁRIO

1. Introdução às Consultas Avançadas
2. Filtragem de Dados com WHERE
 - 2.1. Sintaxe Básica
 - 2.2. Operadores de Comparação
 - 2.3. Combinando Condições
3. Operador LIKE - Busca por Padrões
 - 3.1. Curingas % e _
 - 3.2. Exemplos Práticos
4. Operador BETWEEN - Intervalos
5. Operador IN - Listas de Valores
6. Trabalhando com Valores Nulos (IS NULL)
7. Ordenação de Resultados (ORDER BY)
 - 7.1. Ordem Crescente e Decrescente
 - 7.2. Ordenação por Múltiplas Colunas
8. Limitação de Resultados (LIMIT)
 - 8.1. LIMIT Básico
 - 8.2. OFFSET para Paginação
9. Funções de Agregação
 - 9.1. COUNT - Contagem
 - 9.2. SUM - Soma
 - 9.3. AVG - Média
 - 9.4. MIN e MAX - Valores Extremos
 - 9.5. Combinando Funções
10. Atividades Práticas Guiadas
11. Referências e Recursos Adicionais

1. INTRODUÇÃO ÀS CONSULTAS AVANÇADAS

Até agora, aprendemos a criar tabelas, inserir dados e fazer consultas básicas com SELECT. Nesta aula, vamos além: aprenderemos a **filtrar**, **ordenar** e realizar **cálculos** sobre nossos dados, transformando informações brutas em conhecimento útil.

O que você aprenderá nesta aula:

- Como filtrar registros específicos usando condições complexas
- Buscar padrões em texto (nomes parciais, emails, etc.)
- Ordenar resultados de diversas formas
- Limitar quantidade de resultados e criar paginação
- Calcular estatísticas (contagens, médias, totais)

Por que isso é importante? Em aplicações reais, raramente queremos todos os dados de uma tabela. Precisamos encontrar informações específicas, apresentá-las de forma organizada e extrair insights através de cálculos. Estas técnicas são fundamentais para desenvolver qualquer aplicação com banco de dados.

2. FILTRAGEM DE DADOS COM WHERE

A cláusula **WHERE** é o coração da filtragem em SQL. Ela permite selecionar apenas os registros que atendem a determinadas condições, ignorando o restante.

2.1. Sintaxe Básica

```
# Estrutura básica do SELECT com WHERE
SELECT coluna1, coluna2, ...
FROM nome_tabela
WHERE condição;

# A condição determina quais linhas serão retornadas
# Apenas registros onde a condição é VERDADEIRA aparecem no resultado
```

Exemplo Introdutório

Vamos começar com um exemplo simples. Imagine que temos uma tabela de alunos e queremos ver apenas os alunos de uma cidade específica:

```
import sqlite3

# Conectar ao banco de dados
conexao = sqlite3.connect('escola.db')
cursor = conexao.cursor()

# Consulta SEM filtro - retorna TODOS os alunos
cursor.execute('SELECT nome, cidade FROM alunos')
print("TODOS OS ALUNOS:")
for aluno in cursor.fetchall():
    print(f" {aluno[0]} - {aluno[1]}")

print("\n" + "="*50 + "\n")

# Consulta COM filtro - retorna apenas alunos de Salvador
cursor.execute('''
SELECT nome, cidade
FROM alunos
WHERE cidade = 'Salvador'
''')

print("APENAS ALUNOS DE SALVADOR:")
for aluno in cursor.fetchall():
    print(f" {aluno[0]} - {aluno[1]}")

conexao.close()
```

Importante: A cláusula WHERE sempre vem **depois** do FROM e **antes** de ORDER BY ou LIMIT (que veremos mais adiante).

2.2. Operadores de Comparação

SQL oferece diversos operadores para criar condições. Vejamos os principais:

Operador	Significado	Exemplo	Resultado
=	Igual a	idade = 20	Idade exatamente 20
!=	Diferente de	cidade != 'Salvador'	Todas exceto Salvador
<>	Diferente de	nota <> 10	Notas diferentes de 10
<	Menor que	idade < 18	Menores de 18
>	Maior que	nota > 7	Notas acima de 7
<=	Menor ou igual	preco <= 100	Até 100 reais
>=	Maior ou igual	idade >= 21	21 anos ou mais

Exemplos Práticos com Operadores

```
import sqlite3

conexao = sqlite3.connect('escola.db')
cursor = conexao.cursor()

# Exemplo 1: Alunos maiores de 20 anos
cursor.execute('''
SELECT nome, idade
FROM alunos
WHERE idade > 20
''')
print("Alunos com mais de 20 anos:")
for aluno in cursor.fetchall():
    print(f" {aluno[0]} - {aluno[1]} anos")

# Exemplo 2: Alunos com nota maior ou igual a 7
cursor.execute('''
SELECT nome, nota
FROM alunos
WHERE nota >= 7.0
ORDER BY nota DESC
''')
print("\nAlunos aprovados (nota >= 7):")
for aluno in cursor.fetchall():
    print(f" {aluno[0]} - Nota: {aluno[1]}")

# Exemplo 3: Produtos com preço menor que 50
cursor.execute('''
SELECT nome, preco
FROM produtos
WHERE preco < 50.00
''')
print("\nProdutos abaixo de R$ 50:")
for produto in cursor.fetchall():
    print(f" {produto[0]} - R$ {produto[1]:.2f}")

conexao.close()
```

2.3. Combinando Condições

Podemos combinar múltiplas condições usando os operadores lógicos **AND** e **OR**:

- **AND** - Todas as condições devem ser verdadeiras
- **OR** - Pelo menos uma condição deve ser verdadeira
- **NOT** - Inverte o resultado da condição

```
import sqlite3

conexao = sqlite3.connect('escola.db')
cursor = conexao.cursor()

# AND - Ambas condições devem ser verdadeiras
cursor.execute('''
SELECT nome, idade, cidade
FROM alunos
WHERE idade >= 18 AND cidade = 'Salvador'
''')
print("Alunos maiores de idade DE SALVADOR:")
for aluno in cursor.fetchall():
    print(f" {aluno[0]} - {aluno[1]} anos - {aluno[2]}")

# OR - Pelo menos uma condição deve ser verdadeira
cursor.execute('''
SELECT nome, cidade, nota
FROM alunos
WHERE cidade = 'Salvador' OR nota >= 9.0
''')
print("\nAlunos de Salvador OU com nota excelente:")
for aluno in cursor.fetchall():
    print(f" {aluno[0]} - {aluno[1]} - Nota: {aluno[2]}")

# Combinação complexa com parênteses
cursor.execute('''
SELECT nome, idade, cidade, nota
FROM alunos
WHERE (cidade = 'Salvador' OR cidade = 'São Paulo')
    AND idade >= 20
    AND nota > 7.0
''')
print("\nAlunos de Salvador/SP, maiores de 20 e nota > 7:")
for aluno in cursor.fetchall():
    print(f" {aluno[0]} - {aluno[1]} anos - {aluno[2]} - Nota: {aluno[3]}")

conexao.close()
```

Dica: Use parênteses para deixar clara a ordem de avaliação das condições, especialmente quando mistura AND e OR.

3. OPERADOR LIKE - BUSCA POR PADRÕES

O operador **LIKE** permite buscar padrões em campos de texto. É extremamente útil quando você não sabe o valor exato que está procurando, mas conhece parte dele.

3.1. Curingas % e _

O LIKE usa dois caracteres especiais chamados **curingas**:

Curinga	Significado	Exemplo	Match
%	Zero ou mais caracteres	'A%'	Ana, Antonio, André, A
_	Exatamente um caractere	'_aria'	Maria, Daria (5 letras)

Padrões Comuns com LIKE

Padrão	Descrição	Exemplos que fazem match
'A%'	Começa com A	Ana, Antonio, André, Amanda
'%silva%'	Contém silva (maiúsc/minúsc)	João Silva, Silvania, Silva Costa
'%a'	Termina com a	Maria, Carla, Paula, Ana
'M%a'	Começa com M e termina com a	Maria, Marta, Mariana, Monalisa
'___'	Exatamente 3 caracteres	Ana, Bob, Max (não João)
'%@gmail.com'	Email do Gmail	user@gmail.com, test@gmail.com

3.2. Exemplos Práticos com LIKE

```
import sqlite3

conexao = sqlite3.connect('escola.db')
cursor = conexao.cursor()

# Buscar alunos cujo nome começa com 'M'
cursor.execute('''
SELECT nome, email
FROM alunos
WHERE nome LIKE 'M%'
''')
print("Alunos cujo nome começa com M:")
for aluno in cursor.fetchall():
    print(f" {aluno[0]} - {aluno[1]}")

# Buscar alunos com 'Silva' no nome
cursor.execute('''
SELECT nome
FROM alunos
WHERE nome LIKE '%Silva%'
''')
print("\nAlunos com 'Silva' no nome:")
for aluno in cursor.fetchall():
    print(f" {aluno[0]}")

# Buscar emails do domínio gmail
cursor.execute('''
SELECT nome, email
FROM alunos
WHERE email LIKE '%@gmail.com'
''')
print("\nAlunos com email do Gmail:")
for aluno in cursor.fetchall():
    print(f" {aluno[0]} - {aluno[1]}")

# Buscar nomes com exatamente 5 letras
cursor.execute('''
SELECT nome
FROM alunos
WHERE nome LIKE '_____'
''')
print("\nNomes com exatamente 5 letras:")
for aluno in cursor.fetchall():
    print(f" {aluno[0]}")

# Combinar LIKE com outras condições
cursor.execute('''
SELECT nome, cidade, nota
FROM alunos
WHERE nome LIKE 'A%'
      AND cidade = 'Salvador'
      AND nota >= 8.0
''')
print("\nAlunos de Salvador, nome com A, nota >= 8:")
for aluno in cursor.fetchall():
    print(f" {aluno[0]} - {aluno[1]} - Nota: {aluno[2]}")

conexao.close()
```

Atenção: O LIKE pode ser mais lento que comparações exatas (=) em tabelas grandes. Use-o quando realmente precisar buscar padrões, não valores exatos.

4. OPERADOR BETWEEN - INTERVALOS

O operador **BETWEEN** verifica se um valor está dentro de um intervalo (inclusivo). É uma forma mais clara e concisa que usar $\geq$ e $\leq$.

Sintaxe

```
SELECT colunas
FROM tabela
WHERE coluna BETWEEN valor_inicial AND valor_final;

-- É equivalente a:
WHERE coluna >= valor_inicial AND coluna <= valor_final

-- Note que BETWEEN é INCLUSIVO (inclui os valores das extremidades)
```

Exemplos com Diferentes Tipos de Dados

```
import sqlite3

conexao = sqlite3.connect('escola.db')
cursor = conexao.cursor()

# BETWEEN com números inteiros
cursor.execute('''
SELECT nome, idade
FROM alunos
WHERE idade BETWEEN 18 AND 25
ORDER BY idade
''')
print("Alunos entre 18 e 25 anos (inclusivo):")
for aluno in cursor.fetchall():
    print(f" {aluno[0]} - {aluno[1]} anos")

# BETWEEN com números decimais
cursor.execute('''
SELECT nome, nota
FROM alunos
WHERE nota BETWEEN 7.0 AND 9.0
ORDER BY nota DESC
''')
print("\nAlunos com nota entre 7.0 e 9.0:")
for aluno in cursor.fetchall():
    print(f" {aluno[0]} - Nota: {aluno[1]}")

# BETWEEN com datas
cursor.execute('''
SELECT nome, data_nascimento
FROM alunos
WHERE data_nascimento BETWEEN '2000-01-01' AND '2002-12-31'
ORDER BY data_nascimento
''')
print("\nAlunos nascidos entre 2000 e 2002:")
for aluno in cursor.fetchall():
    print(f" {aluno[0]} - Nascimento: {aluno[1]}")

# BETWEEN com texto (ordem alfabética)
cursor.execute('''
SELECT nome
FROM alunos
WHERE nome BETWEEN 'A' AND 'M'
ORDER BY nome
''')
print("\nAlunos com nome de A até M:")
for aluno in cursor.fetchall():
    print(f" {aluno[0]}")

# NOT BETWEEN (fora do intervalo)
cursor.execute('''
SELECT nome, idade
FROM alunos
WHERE idade NOT BETWEEN 20 AND 30
ORDER BY idade
''')
print("\nAlunos FORA da faixa 20-30 anos:")
```

```
for aluno in cursor.fetchall():  
    print(f" {aluno[0]} - {aluno[1]} anos")  
conexao.close()
```

5. OPERADOR IN - LISTAS DE VALORES

O operador **IN** verifica se um valor está presente em uma lista de valores especificados. É muito mais limpo que usar múltiplos OR.

Comparação: IN vs OR

```
# Usando múltiplos OR (verboso e repetitivo)
SELECT nome, cidade
FROM alunos
WHERE cidade = 'Salvador'
      OR cidade = 'São Paulo'
      OR cidade = 'Rio de Janeiro'
      OR cidade = 'Brasília';

# Usando IN (muito mais limpo)
SELECT nome, cidade
FROM alunos
WHERE cidade IN ('Salvador', 'São Paulo', 'Rio de Janeiro', 'Brasília');
```

Exemplos Práticos

```
import sqlite3

conexao = sqlite3.connect('escola.db')
cursor = conexao.cursor()

# IN com strings
cursor.execute('''
SELECT nome, cidade
FROM alunos
WHERE cidade IN ('Salvador', 'São Paulo', 'Rio de Janeiro')
ORDER BY cidade, nome
''')
print("Alunos das capitais selecionadas:")
cidade_atual = None
for aluno in cursor.fetchall():
    if cidade_atual != aluno[1]:
        cidade_atual = aluno[1]
        print(f"\n{cidade_atual}:")
    print(f"  {aluno[0]}")

# IN com números
cursor.execute('''
SELECT nome, id_turma
FROM alunos
WHERE id_turma IN (1, 3, 5)
ORDER BY id_turma
''')
print("\nAlunos das turmas 1, 3 e 5:")
for aluno in cursor.fetchall():
    print(f"  {aluno[0]} - Turma {aluno[1]}")

# NOT IN (não está na lista)
cursor.execute('''
SELECT nome, cidade
FROM alunos
WHERE cidade NOT IN ('Salvador', 'São Paulo')
ORDER BY nome
''')
print("\nAlunos que NÃO são de Salvador nem São Paulo:")
for aluno in cursor.fetchall():
    print(f"  {aluno[0]} - {aluno[1]}")

# IN com subconsulta (avançado)
cursor.execute('''
SELECT nome, nota
FROM alunos
WHERE id_turma IN (
    SELECT id_turma
    FROM turmas
    WHERE periodo = 'Manhã'
)
''')
print("\nAlunos de turmas do período da manhã:")
```

```
for aluno in cursor.fetchall():  
    print(f" {aluno[0]} - Nota: {aluno[1]}")  
conexao.close()
```

6. TRABALHANDO COM VALORES NULOS (IS NULL)

Em bancos de dados, **NULL** representa a ausência de valor. É diferente de zero, string vazia ou falso. Para verificar se um campo é NULL, usamos **IS NULL** ou **IS NOT NULL**.

IMPORTANTE: NUNCA use `= NULL` ou `!= NULL`. Isso não funciona! Sempre use **IS NULL** ou **IS NOT NULL**.

Por que `= NULL` não funciona?

NULL significa 'desconhecido'. Quando comparamos algo com um valor desconhecido, o resultado também é desconhecido (não é verdadeiro nem falso). Por isso, precisamos dos operadores especiais **IS NULL** e **IS NOT NULL**.

```
import sqlite3

conexao = sqlite3.connect('escola.db')
cursor = conexao.cursor()

# ■ ERRADO - não funciona!
cursor.execute('SELECT nome FROM alunos WHERE email = NULL')
print("Usando = NULL (ERRADO):", cursor.fetchall()) # Retorna lista vazia

# ✓ CORRETO - use IS NULL
cursor.execute('SELECT nome, email FROM alunos WHERE email IS NULL')
print("\nAlunos SEM email cadastrado (IS NULL):")
for aluno in cursor.fetchall():
    print(f" {aluno[0]} - Email: {aluno[1]}")

# Alunos COM email
cursor.execute('''
SELECT nome, email
FROM alunos
WHERE email IS NOT NULL
ORDER BY nome
''')
print("\nAlunos COM email cadastrado:")
for aluno in cursor.fetchall():
    print(f" {aluno[0]} - {aluno[1]}")

# Contar alunos com e sem email
cursor.execute('SELECT COUNT(*) FROM alunos WHERE email IS NULL')
sem_email = cursor.fetchone()[0]

cursor.execute('SELECT COUNT(*) FROM alunos WHERE email IS NOT NULL')
com_email = cursor.fetchone()[0]

print(f"\nEstatísticas:")
print(f" Alunos sem email: {sem_email}")
print(f" Alunos com email: {com_email}")
print(f" Total: {sem_email + com_email}")

# Combinar IS NULL com outras condições
cursor.execute('''
SELECT nome, cidade, email
FROM alunos
WHERE cidade = 'Salvador'
AND email IS NULL
''')
print("\nAlunos de Salvador SEM email:")
for aluno in cursor.fetchall():
    print(f" {aluno[0]} - {aluno[1]}")

conexao.close()
```

7. ORDENAÇÃO DE RESULTADOS (ORDER BY)

A cláusula **ORDER BY** ordena os resultados de uma consulta. Sem ela, a ordem dos registros é imprevisível e pode mudar entre execuções.

7.1. Ordem Crescente e Decrescente

```
SELECT colunas
FROM tabela
ORDER BY coluna [ASC|DESC];

-- ASC: ordem crescente (padrão, pode ser omitido)
-- DESC: ordem decrescente

import sqlite3

conexao = sqlite3.connect('escola.db')
cursor = conexao.cursor()

# Ordem alfabética (A-Z) - ASC é padrão
cursor.execute('''
SELECT nome, idade
FROM alunos
ORDER BY nome ASC
''')
print("Alunos em ordem alfabética (A-Z):")
for i, aluno in enumerate(cursor.fetchall(), 1):
    print(f" {i}. {aluno[0]} - {aluno[1]} anos")

# Ordem alfabética reversa (Z-A)
cursor.execute('''
SELECT nome
FROM alunos
ORDER BY nome DESC
''')
print("\nAlunos em ordem reversa (Z-A):")
for aluno in cursor.fetchall():
    print(f" {aluno[0]}")

# Ordenar por idade (do mais jovem ao mais velho)
cursor.execute('''
SELECT nome, idade
FROM alunos
ORDER BY idade ASC
''')
print("\nDo mais jovem ao mais velho:")
for aluno in cursor.fetchall():
    print(f" {aluno[0]} - {aluno[1]} anos")

# Ordenar por nota (maior para menor)
cursor.execute('''
SELECT nome, nota
FROM alunos
WHERE nota IS NOT NULL
ORDER BY nota DESC
''')
print("\nRanking de notas (maior para menor):")
for i, aluno in enumerate(cursor.fetchall(), 1):
    print(f" {i}º lugar: {aluno[0]} - Nota {aluno[1]}")

conexao.close()
```

7.2. Ordenação por Múltiplas Colunas

Você pode ordenar por várias colunas. A segunda coluna é usada como 'desempate' quando os valores da primeira coluna são iguais.

```
import sqlite3

conexao = sqlite3.connect('escola.db')
cursor = conexao.cursor()

# Ordenar por cidade, depois por nome dentro de cada cidade
cursor.execute('''
SELECT nome, cidade, idade
FROM alunos
ORDER BY cidade ASC, nome ASC
''')

print("Alunos agrupados por cidade (alfabético):")
cidade_atual = None
for aluno in cursor.fetchall():
    if cidade_atual != aluno[1]:
        cidade_atual = aluno[1]
        print(f"\n{cidade_atual}:")
    print(f"  - {aluno[0]} ({aluno[2]} anos)")

# Ordenar por nota (decrescente), depois por nome (crescente)
cursor.execute('''
SELECT nome, nota
FROM alunos
WHERE nota IS NOT NULL
ORDER BY nota DESC, nome ASC
''')
print("\n\nRanking: Nota DESC, depois Nome ASC (desempate):")
posicao = 1
for aluno in cursor.fetchall():
    print(f"  {posicao}º - {aluno[0]} - Nota: {aluno[1]}")
    posicao += 1

# Exemplo com 3 níveis de ordenação
cursor.execute('''
SELECT nome, cidade, idade, nota
FROM alunos
ORDER BY cidade ASC, nota DESC, nome ASC
''')
print("\n\nOrdenação tripla (Cidade ↑, Nota ↓, Nome ↑):")
cidade_atual = None
for aluno in cursor.fetchall():
    if cidade_atual != aluno[1]:
        cidade_atual = aluno[1]
        print(f"\n{cidade_atual}:")
    nota_str = f"{aluno[3]:.1f}" if aluno[3] is not None else "N/A"
    print(f"  {aluno[0]} - {aluno[2]} anos - Nota: {nota_str}")

conexao.close()
```

Dica: A ordem das colunas no ORDER BY importa! A primeira coluna é a ordenação principal, a segunda é secundária (desempate), e assim por diante.

8. LIMITAÇÃO DE RESULTADOS (LIMIT)

A cláusula **LIMIT** restringe o número de registros retornados. É essencial para visualizar amostras de dados, criar rankings (top 10) e implementar paginação.

8.1. LIMIT Básico

```
import sqlite3

conexao = sqlite3.connect('escola.db')
cursor = conexao.cursor()

# Retornar apenas os 5 primeiros alunos
cursor.execute('''
SELECT nome, idade
FROM alunos
LIMIT 5
''')
print("Primeiros 5 alunos:")
for aluno in cursor.fetchall():
    print(f" {aluno[0]} - {aluno[1]} anos")

# Top 3 melhores notas
cursor.execute('''
SELECT nome, nota
FROM alunos
WHERE nota IS NOT NULL
ORDER BY nota DESC
LIMIT 3
''')
print("\nTop 3 melhores notas:")
for i, aluno in enumerate(cursor.fetchall(), 1):
    print(f" {i}º lugar: {aluno[0]} - {aluno[1]}")

# 5 alunos mais velhos
cursor.execute('''
SELECT nome, idade
FROM alunos
ORDER BY idade DESC
LIMIT 5
''')
print("\n5 alunos mais velhos:")
for aluno in cursor.fetchall():
    print(f" {aluno[0]} - {aluno[1]} anos")

conexao.close()
```

8.2. OFFSET para Paginação

O **OFFSET** (ou deslocamento) pula um número específico de registros antes de começar a retornar resultados. Combinado com **LIMIT**, permite criar paginação.

```
import sqlite3

def buscar_pagina(numero_pagina, itens_por_pagina=5):
    """
    Busca uma página específica de alunos.

    Args:
        numero_pagina: Número da página (começa em 1)
        itens_por_pagina: Quantos itens por página

    Returns:
        Lista de alunos da página solicitada
    """
    conexao = sqlite3.connect('escola.db')
    cursor = conexao.cursor()

    # Calcular o OFFSET
    offset = (numero_pagina - 1) * itens_por_pagina

    cursor.execute('''
    SELECT nome, email, idade
    FROM alunos
    ORDER BY nome
    LIMIT ? OFFSET ?
    ''', (itens_por_pagina, offset))

    resultados = cursor.fetchall()
    conexao.close()

    return resultados

# Contar total de alunos
conexao = sqlite3.connect('escola.db')
cursor = conexao.cursor()
cursor.execute('SELECT COUNT(*) FROM alunos')
total_alunos = cursor.fetchone()[0]
conexao.close()

itens_por_pagina = 5
total_paginas = (total_alunos + itens_por_pagina - 1) // itens_por_pagina

print(f"Total de alunos: {total_alunos}")
print(f"Itens por página: {itens_por_pagina}")
print(f"Total de páginas: {total_paginas}")
print("="*60)

# Exibir cada página
for pagina in range(1, total_paginas + 1):
    print(f"\nPÁGINA {pagina} de {total_paginas}:")
    print("-" * 60)

    alunos = buscar_pagina(pagina, itens_por_pagina)

    for i, aluno in enumerate(alunos, 1):
        email = aluno[1] if aluno[1] else "Sem email"
        print(f"  {i}. {aluno[0]} - {email} - {aluno[2]} anos")

    if pagina < total_paginas:
        input("\nPressione Enter para próxima página...")
```

Fórmula para paginação:

$\text{OFFSET} = (\text{número_da_página} - 1) \times \text{itens_por_página}$

9. FUNÇÕES DE AGREGAÇÃO

Funções de agregação realizam cálculos sobre conjuntos de valores e retornam um único resultado. São fundamentais para criar relatórios e análises estatísticas.

Função	O que faz	Exemplo de Uso
COUNT()	Conta registros ou valores não-nulos	Quantos alunos?
SUM()	Soma valores numéricos	Total de vendas
AVG()	Calcula média aritmética	Média de idades
MIN()	Retorna o menor valor	Nota mais baixa
MAX()	Retorna o maior valor	Produto mais caro

9.1. COUNT - Contagem

COUNT conta o número de registros. Use **COUNT(*)** para contar todos os registros ou **COUNT(coluna)** para contar valores não-nulos.

```
import sqlite3

conexao = sqlite3.connect('escola.db')
cursor = conexao.cursor()

# Contar total de alunos
cursor.execute('SELECT COUNT(*) FROM alunos')
total = cursor.fetchone()[0]
print(f"Total de alunos cadastrados: {total}")

# Contar alunos com email (ignora NULL)
cursor.execute('SELECT COUNT(email) FROM alunos')
com_email = cursor.fetchone()[0]
print(f"Alunos com email cadastrado: {com_email}")

# Contar com filtro
cursor.execute('''
SELECT COUNT(*) FROM alunos
WHERE cidade = 'Salvador'
''')
salvador = cursor.fetchone()[0]
print(f"Alunos de Salvador: {salvador}")

# Contar alunos aprovados (nota >= 7)
cursor.execute('''
SELECT COUNT(*) FROM alunos
WHERE nota >= 7.0
''')
aprovados = cursor.fetchone()[0]
print(f"Alunos aprovados: {aprovados}")

# Combinar COUNT com outras funções
cursor.execute('''
SELECT
    COUNT(*) as total,
    COUNT(email) as com_email,
    COUNT(*) - COUNT(email) as sem_email
FROM alunos
''')
stats = cursor.fetchone()
print(f"\nResumo:")
print(f"  Total: {stats[0]}")
print(f"  Com email: {stats[1]}")
print(f"  Sem email: {stats[2]}")

conexao.close()
```

9.2. SUM - Soma

SUM calcula a soma de todos os valores de uma coluna numérica.

```
import sqlite3

conexao = sqlite3.connect('escola.db')
cursor = conexao.cursor()

# Soma de todas as notas
cursor.execute('SELECT SUM(nota) FROM alunos WHERE nota IS NOT NULL')
soma_notas = cursor.fetchone()[0]
print(f"Soma de todas as notas: {soma_notas:.2f}")

# Soma de idades
cursor.execute('SELECT SUM(idade) FROM alunos')
soma_idades = cursor.fetchone()[0]
print(f"Soma de todas as idades: {soma_idades}")

# Exemplo com vendas (se tivéssemos tabela de vendas)
cursor.execute('''
SELECT SUM(preco * quantidade) as receita_total
FROM vendas
WHERE data >= '2024-01-01'
''')
# receita = cursor.fetchone()[0]

# SUM com filtro
cursor.execute('''
SELECT SUM(nota) FROM alunos
WHERE cidade = 'Salvador' AND nota IS NOT NULL
''')
soma_salvador = cursor.fetchone()[0]
print(f"Soma das notas (Salvador): {soma_salvador:.2f}")

conexao.close()
```

9.3. AVG - Média

AVG calcula a média aritmética dos valores. Valores NULL são ignorados no cálculo.

```
import sqlite3

conexao = sqlite3.connect('escola.db')
cursor = conexao.cursor()

# Média de idade de todos os alunos
cursor.execute('SELECT AVG(idade) FROM alunos')
media_idade = cursor.fetchone()[0]
print(f"Média de idade: {media_idade:.1f} anos")

# Média de notas
cursor.execute('SELECT AVG(nota) FROM alunos WHERE nota IS NOT NULL')
media_notas = cursor.fetchone()[0]
print(f"Média geral de notas: {media_notas:.2f}")

# Média de notas por cidade
cursor.execute('''
SELECT cidade, AVG(nota) as media
FROM alunos
WHERE nota IS NOT NULL
GROUP BY cidade
ORDER BY media DESC
''')
print("\nMédia de notas por cidade:")
for cidade in cursor.fetchall():
    print(f" {cidade[0]}: {cidade[1]:.2f}")

# Média com filtro
cursor.execute('''
SELECT AVG(nota) FROM alunos
WHERE nota >= 7.0
''')
media_aprovados = cursor.fetchone()[0]
print(f"\nMédia dos alunos aprovados: {media_aprovados:.2f}")

conexao.close()
```

9.4. MIN e MAX - Valores Extremos

MIN retorna o menor valor e MAX retorna o maior valor de uma coluna.

```
import sqlite3

conexao = sqlite3.connect('escola.db')
cursor = conexao.cursor()

# Menor e maior idade
cursor.execute('SELECT MIN(idade), MAX(idade) FROM alunos')
min_idade, max_idade = cursor.fetchone()
print(f"Idade mínima: {min_idade} anos")
print(f"Idade máxima: {max_idade} anos")
print(f"Diferença: {max_idade - min_idade} anos")

# Menor e maior nota
cursor.execute('''
SELECT MIN(nota), MAX(nota)
FROM alunos
WHERE nota IS NOT NULL
''')
min_nota, max_nota = cursor.fetchone()
print(f"\nMenor nota: {min_nota}")
print(f"Maior nota: {max_nota}")

# Encontrar o aluno com a maior nota
cursor.execute('''
SELECT nome, nota
FROM alunos
WHERE nota = (SELECT MAX(nota) FROM alunos)
''')
melhor_aluno = cursor.fetchone()
print(f"\nMelhor aluno: {melhor_aluno[0]} - Nota: {melhor_aluno[1]}")

# Encontrar o aluno com a menor nota
cursor.execute('''
SELECT nome, nota
FROM alunos
WHERE nota = (SELECT MIN(nota) FROM alunos WHERE nota IS NOT NULL)
''')
pior_aluno = cursor.fetchone()
print(f"Menor nota: {pior_aluno[0]} - Nota: {pior_aluno[1]}")

# MIN/MAX com datas
cursor.execute('''
SELECT MIN(data_nascimento), MAX(data_nascimento)
FROM alunos
''')
mais_velho, mais_novo = cursor.fetchone()
print(f"\nAluno mais velho nasceu em: {mais_velho}")
print(f"Aluno mais novo nasceu em: {mais_novo}")

conexao.close()
```

9.5. Combinando Funções de Agregação

Você pode usar múltiplas funções de agregação na mesma consulta para criar relatórios estatísticos completos:

```
import sqlite3

def gerar_relatorio_estatistico():
    """Gera um relatório estatístico completo dos alunos."""

    conexao = sqlite3.connect('escola.db')
    cursor = conexao.cursor()

    # Estatísticas gerais
    cursor.execute('''
    SELECT
        COUNT(*) as total_alunos,
        COUNT(email) as com_email,
        AVG(idade) as media_idade,
        MIN(idade) as menor_idade,
        MAX(idade) as maior_idade,
        AVG(nota) as media_nota,
        MIN(nota) as menor_nota,
        MAX(nota) as maior_nota,
        SUM(nota) as soma_notas
    FROM alunos
    WHERE nota IS NOT NULL
    ''')

    stats = cursor.fetchone()

    print("="*60)
    print("RELATÓRIO ESTATÍSTICO - ESCOLA")
    print("="*60)

    print("\nDados Gerais:")
    print(f" Total de alunos: {stats[0]}")
    print(f" Alunos com email: {stats[1]} ({stats[1]/stats[0]*100:.1f}%)")

    print("\nEstatísticas de Idade:")
    print(f" Média: {stats[2]:.1f} anos")
    print(f" Mínima: {stats[3]} anos")
    print(f" Máxima: {stats[4]} anos")
    print(f" Amplitude: {stats[4] - stats[3]} anos")

    print("\nEstatísticas de Notas:")
    print(f" Média: {stats[5]:.2f}")
    print(f" Menor nota: {stats[6]}")
    print(f" Maior nota: {stats[7]}")
    print(f" Amplitude: {stats[7] - stats[6]}")
    print(f" Soma total: {stats[8]:.2f}")

    # Estatísticas por cidade
    cursor.execute('''
    SELECT
        cidade,
        COUNT(*) as total,
        AVG(nota) as media,
        MIN(nota) as menor,
        MAX(nota) as maior
    FROM alunos
    WHERE nota IS NOT NULL
    GROUP BY cidade
    ORDER BY media DESC
    ''')

    print("\nEstatísticas por Cidade:")
    print(f" {'Cidade':<20} {'Total':<8} {'Média':<8} {'Min':<8} {'Max':<8}")
    print(" " + "-"*56)

    for row in cursor.fetchall():
        print(f" {row[0]:<20} {row[1]:<8} {row[2]:<8.2f} {row[3]:<8.1f} {row[4]:<8.1f}")

    # Taxa de aprovação
    cursor.execute('''
    SELECT
        COUNT(*) as total,
        SUM(CASE WHEN nota >= 7.0 THEN 1 ELSE 0 END) as aprovados,
        SUM(CASE WHEN nota < 7.0 THEN 1 ELSE 0 END) as reprovados
    ''')
```

```
FROM alunos
WHERE nota IS NOT NULL
'''

aprovacao = cursor.fetchone()
taxa = (aprovacao[1] / aprovacao[0] * 100) if aprovacao[0] > 0 else 0

print("\nTaxa de Aprovação (nota >= 7.0):")
print(f"  Total avaliados: {aprovacao[0]}")
print(f"  Aprovados: {aprovacao[1]} ({aprovacao[1]/aprovacao[0]*100:.1f}%)")
print(f"  Reprovados: {aprovacao[2]} ({aprovacao[2]/aprovacao[0]*100:.1f}%)")

print("\n" + "="*60)

conexao.close()

# Executar relatório
gerar_relatorio_estatistico()
```

10. ATIVIDADES PRÁTICAS GUIADAS

Agora é hora de colocar em prática tudo o que aprendemos! Vamos trabalhar com três atividades progressivas usando o banco de dados da escola.

Preparação: Criar o Banco de Dados

Primeiro, vamos criar um banco de dados completo com dados de exemplo:

```
import sqlite3
from datetime import date, timedelta
import random

# Criar e popular banco de dados
conexao = sqlite3.connect('escola.db')
cursor = conexao.cursor()

# Limpar tabelas se existirem
cursor.execute('DROP TABLE IF EXISTS alunos')
cursor.execute('DROP TABLE IF EXISTS turmas')

# Criar tabela de turmas
cursor.execute('''
CREATE TABLE turmas (
    id_turma INTEGER PRIMARY KEY AUTOINCREMENT,
    nome_turma VARCHAR(50) NOT NULL,
    curso VARCHAR(50) NOT NULL,
    ano INTEGER NOT NULL,
    periodo VARCHAR(20)
)''')

# Criar tabela de alunos
cursor.execute('''
CREATE TABLE alunos (
    id_aluno INTEGER PRIMARY KEY AUTOINCREMENT,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(100),
    data_nascimento DATE,
    cidade VARCHAR(50),
    nota REAL,
    id_turma INTEGER,
    FOREIGN KEY (id_turma) REFERENCES turmas(id_turma)
)''')

# Inserir turmas
turmas = [
    ('Turma A', 'Python Básico', 2024, 'Manhã'),
    ('Turma B', 'Python Avançado', 2024, 'Tarde'),
    ('Turma C', 'Data Science', 2024, 'Noite'),
    ('Turma D', 'Web Development', 2024, 'Manhã'),
    ('Turma E', 'Machine Learning', 2024, 'Tarde')
]

cursor.executemany('''
INSERT INTO turmas (nome_turma, curso, ano, periodo)
VALUES (?, ?, ?, ?)
''', turmas)

# Inserir alunos com dados variados
alunos = [
    ('Ana Silva', 'ana.silva@gmail.com', '2000-03-15', 'Salvador', 8.5, 1),
    ('Bruno Costa', 'bruno@gmail.com', '1998-07-22', 'São Paulo', 7.2, 1),
    ('Carla Souza', 'carla@hotmail.com', '2001-11-10', 'Rio de Janeiro', 9.1, 2),
    ('Daniel Lima', None, '1999-01-30', 'Salvador', 6.8, 2),
    ('Elena Ferreira', 'elena@gmail.com', '2002-09-05', 'Brasília', 8.9, 3),
    ('Felipe Martins', 'felipe@outlook.com', '1997-12-18', 'São Paulo', 7.5, 3),
    ('Gabriela Rocha', 'gabi@gmail.com', '2000-05-25', 'Salvador', 9.5, 4),
    ('Henrique Alves', 'henrique@gmail.com', '2001-08-14', 'Rio de Janeiro', 6.5, 4),
    ('Isabela Dias', None, '1999-04-03', 'Salvador', 8.2, 1),
    ('João Pedro', 'joao@gmail.com', '2000-10-20', 'Brasília', 7.8, 2),
    ('Karen Santos', 'karen@gmail.com', '2001-02-28', 'São Paulo', 9.0, 5),
    ('Lucas Oliveira', None, '1998-11-12', 'Salvador', 7.1, 5),
    ('Mariana Costa', 'mari@hotmail.com', '2002-06-07', 'Rio de Janeiro', 8.3, 3),
    ('Nicolas Pereira', 'nicolas@gmail.com', '1999-09-19', 'Brasília', 6.9, 4),
]
```

```
        ('Olivia Lima', 'olivia@gmail.com', '2000-12-01', 'São Paulo', 8.7, 5)
    ]

    cursor.executemany('''
INSERT INTO alunos (nome, email, data_nascimento, cidade, nota, id_turma)
VALUES (?, ?, ?, ?, ?, ?)
''', alunos)

    conexao.commit()
    print("Banco de dados criado com sucesso!")
    print(f"- {len(turmas)} turmas inseridas")
    print(f"- {len(alunos)} alunos inseridos")

    conexao.close()
```

ATIVIDADE 1: Consultas com Filtros Diversos

Objetivo: Praticar filtragem de dados usando WHERE com diferentes operadores.

Tarefas:

1. Listar todos os alunos de Salvador
2. Encontrar alunos com nota maior ou igual a 8.0
3. Buscar alunos cujo nome começa com 'A'
4. Listar alunos sem email cadastrado
5. Encontrar alunos de Salvador OU São Paulo que tenham nota > 7.0

```
import sqlite3

def atividade1_filtros():
    """Atividade 1: Consultas com diferentes filtros."""

    conexao = sqlite3.connect('escola.db')
    cursor = conexao.cursor()

    # Tarefa 1: Alunos de Salvador
    print("="*60)
    print("TAREFA 1: Alunos de Salvador")
    print("="*60)

    cursor.execute('''
SELECT nome, cidade, nota
FROM alunos
WHERE cidade = 'Salvador'
ORDER BY nome
''')

    for aluno in cursor.fetchall():
        nota_str = f"{aluno[2]:.1f}" if aluno[2] else "N/A"
        print(f" {aluno[0]} - {aluno[1]} - Nota: {nota_str}")

    # Tarefa 2: Notas >= 8.0
    print("\n" + "="*60)
    print("TAREFA 2: Alunos com nota >= 8.0")
    print("="*60)

    cursor.execute('''
SELECT nome, nota
FROM alunos
WHERE nota >= 8.0
ORDER BY nota DESC, nome
''')

    for aluno in cursor.fetchall():
        print(f" {aluno[0]} - Nota: {aluno[1]}")

    # Tarefa 3: Nomes que começam com 'A'
    print("\n" + "="*60)
    print("TAREFA 3: Nomes que começam com 'A'")
    print("="*60)

    cursor.execute('''
SELECT nome, email, cidade
FROM alunos
WHERE nome LIKE 'A%'
ORDER BY nome
''')

    for aluno in cursor.fetchall():
        email = aluno[1] if aluno[1] else "Sem email"
        print(f" {aluno[0]} - {email} - {aluno[2]}")

    # Tarefa 4: Sem email
    print("\n" + "="*60)
    print("TAREFA 4: Alunos sem email cadastrado")
    print("="*60)

    cursor.execute('''
SELECT nome, cidade
FROM alunos
WHERE email IS NULL
ORDER BY nome
''')
```

```

sem_email = cursor.fetchall()
for aluno in sem_email:
    print(f" {aluno[0]} - {aluno[1]}")

print(f"\nTotal de alunos sem email: {len(sem_email)}")

# Tarefa 5: Salvador OU São Paulo com nota > 7
print("\n" + "="*60)
print("TAREFA 5: Salvador/SP com nota > 7.0")
print("="*60)

cursor.execute('''
SELECT nome, cidade, nota
FROM alunos
WHERE (cidade = 'Salvador' OR cidade = 'São Paulo')
AND nota > 7.0
ORDER BY nota DESC
''')

for aluno in cursor.fetchall():
    print(f" {aluno[0]} - {aluno[1]} - Nota: {aluno[2]}")

conexao.close()

# Executar atividade
atividade1_filtros()

```

ATIVIDADE 2: Dados Filtrados e Ordenados

Objetivo: Combinar filtros, ordenação e limitação de resultados.

Tarefas:

1. Top 5 melhores notas
2. Alunos em ordem alfabética
3. Agrupar alunos por cidade e ordenar dentro de cada grupo
4. Implementar paginação (páginas de 5 alunos)
5. Buscar alunos de uma faixa de idade específica

```
import sqlite3

def atividade2_ordenacao():
    """Atividade 2: Ordenação e limitação de resultados."""

    conexao = sqlite3.connect('escola.db')
    cursor = conexao.cursor()

    # Tarefa 1: Top 5 melhores notas
    print("="*60)
    print("TAREFA 1: Top 5 Melhores Notas")
    print("="*60)

    cursor.execute('''
    SELECT nome, nota, cidade
    FROM alunos
    WHERE nota IS NOT NULL
    ORDER BY nota DESC
    LIMIT 5
    ''')

    for i, aluno in enumerate(cursor.fetchall(), 1):
        print(f"  {i}º lugar: {aluno[0]} - Nota {aluno[1]} - {aluno[2]}")

    # Tarefa 2: Ordem alfabética
    print("\n" + "="*60)
    print("TAREFA 2: Todos os Alunos (A-Z)")
    print("="*60)

    cursor.execute('''
    SELECT nome, email
    FROM alunos
    ORDER BY nome ASC
    ''')

    for aluno in cursor.fetchall():
        email = aluno[1] if aluno[1] else "Sem email"
        print(f"  {aluno[0]} - {email}")

    # Tarefa 3: Agrupados por cidade
    print("\n" + "="*60)
    print("TAREFA 3: Alunos Agrupados por Cidade")
    print("="*60)

    cursor.execute('''
    SELECT cidade, nome, nota
    FROM alunos
    ORDER BY cidade ASC, nome ASC
    ''')

    cidade_atual = None
    for aluno in cursor.fetchall():
        if cidade_atual != aluno[0]:
            cidade_atual = aluno[0]
            print(f"\n--- {cidade_atual} ---")

            nota_str = f"{aluno[2]:.1f}" if aluno[2] else "N/A"
            print(f"  {aluno[1]} - Nota: {nota_str}")

    # Tarefa 4: Paginação
    print("\n" + "="*60)
    print("TAREFA 4: Paginação (5 por página)")
    print("="*60)

    itens_por_pagina = 5
```

```

# Contar total
cursor.execute('SELECT COUNT(*) FROM alunos')
total = cursor.fetchone()[0]
total_paginas = (total + itens_por_pagina - 1) // itens_por_pagina

for pagina in range(1, total_paginas + 1):
    offset = (pagina - 1) * itens_por_pagina

    cursor.execute('''
        SELECT nome, cidade
        FROM alunos
        ORDER BY nome
        LIMIT ? OFFSET ?
    ''', (itens_por_pagina, offset))

    print(f"\nPágina {pagina}/{total_paginas}:")
    for aluno in cursor.fetchall():
        print(f"    - {aluno[0]} ({aluno[1]})")

# Tarefa 5: Faixa de idade
print("\n" + "="*60)
print("TAREFA 5: Alunos entre 20 e 24 anos")
print("="*60)

cursor.execute('''
    SELECT nome,
           CAST((julianday('now') - julianday(data_nascimento)) / 365.25 AS INTEGER) as idade,
           cidade
    FROM alunos
    WHERE CAST((julianday('now') - julianday(data_nascimento)) / 365.25 AS INTEGER)
          BETWEEN 20 AND 24
    ORDER BY idade, nome
''')

for aluno in cursor.fetchall():
    print(f"    {aluno[0]} - {aluno[1]} anos - {aluno[2]}")

conexao.close()

# Executar atividade
atividade2_ordenacao()

```

ATIVIDADE 3: Análises com Dados Agrupados

Objetivo: Usar funções de agregação para criar relatórios estatísticos.

Tarefas:

1. Calcular estatísticas gerais (total, médias, min/max)
2. Estatísticas por cidade
3. Estatísticas por turma
4. Calcular taxa de aprovação
5. Criar relatório completo formatado

```
import sqlite3

def atividade3_agregacao():
    """Atividade 3: Análises estatísticas com agregação."""

    conexao = sqlite3.connect('escola.db')
    cursor = conexao.cursor()

    # Tarefa 1: Estatísticas gerais
    print("="*70)
    print("TAREFA 1: Estatísticas Gerais")
    print("="*70)

    cursor.execute('''
    SELECT
        COUNT(*) as total,
        COUNT(email) as com_email,
        AVG(nota) as media,
        MIN(nota) as menor,
        MAX(nota) as maior,
        SUM(nota) as soma
    FROM alunos
    WHERE nota IS NOT NULL
    ''')

    stats = cursor.fetchone()
    print(f"Total de alunos: {stats[0]}")
    print(f"Com email cadastrado: {stats[1]} ({stats[1]/stats[0]*100:.1f}%)")
    print(f"Média de notas: {stats[2]:.2f}")
    print(f"Menor nota: {stats[3]}")
    print(f"Maior nota: {stats[4]}")
    print(f"Soma das notas: {stats[5]:.2f}")

    # Tarefa 2: Por cidade
    print("\n" + "="*70)
    print("TAREFA 2: Estatísticas por Cidade")
    print("="*70)

    cursor.execute('''
    SELECT
        cidade,
        COUNT(*) as total,
        AVG(nota) as media,
        MIN(nota) as menor,
        MAX(nota) as maior
    FROM alunos
    WHERE nota IS NOT NULL
    GROUP BY cidade
    ORDER BY media DESC
    ''')

    print(f"{'Cidade':<20} {'Total':<8} {'Média':<10} {'Min':<8} {'Max':<8}")
    print("-" * 70)

    for row in cursor.fetchall():
        print(f"{'row[0]':<20} {'row[1]':<8} {'row[2]':<10.2f} {'row[3]':<8.1f} {'row[4]':<8.1f}")

    # Tarefa 3: Por turma
    print("\n" + "="*70)
    print("TAREFA 3: Estatísticas por Turma")
    print("="*70)

    cursor.execute('''
    SELECT
        t.nome_turma,
        t.curso,
```

```

        COUNT(a.id_aluno) as total,
        AVG(a.nota) as media,
        MIN(a.nota) as menor,
        MAX(a.nota) as maior
    FROM turmas t
    LEFT JOIN alunos a ON t.id_turma = a.id_turma
    WHERE a.nota IS NOT NULL
    GROUP BY t.id_turma, t.nome_turma, t.curso
    ORDER BY media DESC
'''

print(f"{'Turma':<12} {'Curso':<20} {'Alunos':<8} {'Média':<10}")
print("-" * 70)

for row in cursor.fetchall():
    print(f"{row[0]:<12} {row[1]:<20} {row[2]:<8} {row[3]:<10.2f}")

# Tarefa 4: Taxa de aprovação
print("\n" + "="*70)
print("TAREFA 4: Taxa de Aprovação (nota >= 7.0)")
print("="*70)

cursor.execute('''
SELECT
    COUNT(*) as total,
    SUM(CASE WHEN nota >= 7.0 THEN 1 ELSE 0 END) as aprovados,
    SUM(CASE WHEN nota < 7.0 THEN 1 ELSE 0 END) as reprovados
FROM alunos
WHERE nota IS NOT NULL
''')

aprov = cursor.fetchone()
taxa = (aprov[1] / aprov[0] * 100) if aprov[0] > 0 else 0

print(f"Total de alunos avaliados: {aprov[0]}")
print(f"Aprovados: {aprov[1]} ({aprov[1]/aprov[0]*100:.1f}%)")
print(f"Reprovados: {aprov[2]} ({aprov[2]/aprov[0]*100:.1f}%)")

# Tarefa 5: Relatório completo
print("\n" + "="*70)
print("TAREFA 5: Relatório Completo da Escola")
print("="*70)

# Cabeçalho
print("\n■ RELATÓRIO ACADÊMICO - 2024")
print("-" * 70)

# Resumo executivo
cursor.execute('''
SELECT COUNT(*), AVG(nota), MIN(nota), MAX(nota)
FROM alunos WHERE nota IS NOT NULL
''')
resumo = cursor.fetchone()

print(f"\n■ RESUMO EXECUTIVO:")
print(f"    • Total de alunos: {resumo[0]}")
print(f"    • Média geral: {resumo[1]:.2f}")
print(f"    • Amplitude de notas: {resumo[2]:.1f} a {resumo[3]:.1f}")
print(f"    • Taxa de aprovação: {taxa:.1f}%")

# Top performers
print(f"\n■ TOP 3 ALUNOS:")
cursor.execute('''
SELECT nome, nota, cidade
FROM alunos
WHERE nota IS NOT NULL
ORDER BY nota DESC
LIMIT 3
''')

for i, aluno in enumerate(cursor.fetchall(), 1):
    print(f"    {i}º) {aluno[0]} - {aluno[1]} ({aluno[2]})")

# Distribuição por período
print(f"\n■ DISTRIBUIÇÃO POR PERÍODO:")
cursor.execute('''
SELECT t.periodo, COUNT(a.id_aluno) as total
FROM turmas t
LEFT JOIN alunos a ON t.id_turma = a.id_turma
GROUP BY t.periodo
ORDER BY total DESC
''')

```

```
'''  
for periodo in cursor.fetchall():  
    print(f"    • {periodo[0]}: {periodo[1]} alunos")  
print("\n" + "="*70)  
conexao.close()  
  
# Executar atividade  
atividade3_agregacao()
```

11. REFERÊNCIAS E RECURSOS ADICIONAIS

Documentação Oficial

- SQLite Official Documentation: <https://www.sqlite.org/docs.html>
- Python sqlite3 Module: <https://docs.python.org/3/library/sqlite3.html>
- SQL Tutorial (W3Schools): <https://www.w3schools.com/sql/>
- SQLZoo (Exercícios Interativos): <https://sqlzoo.net/>

Próximos Passos no Aprendizado

1. **GROUP BY e HAVING:** Agrupar dados e filtrar grupos
2. **JOINS:** Relacionar múltiplas tabelas (INNER, LEFT, RIGHT, FULL)
3. **Subconsultas:** Consultas dentro de consultas
4. **UPDATE e DELETE:** Modificar e remover dados com segurança
5. **Índices:** Otimizar performance de consultas
6. **Transações:** Garantir integridade em operações complexas
7. **Views:** Criar consultas reutilizáveis
8. **Triggers:** Automatizar ações no banco de dados

Ferramentas Recomendadas

- **DB Browser for SQLite:** Interface gráfica para visualizar e editar bancos SQLite
- **DBeaver:** Gerenciador universal de bancos de dados
- **SQLite Online:** Ferramenta web para testar SQL sem instalação
- **Python DB-API:** Padrão para trabalhar com diferentes bancos em Python

Parabéns por concluir este material! Agora você domina as técnicas essenciais de consultas SQL. Continue praticando, criando projetos próprios e explorando recursos mais avançados. A prática constante é a chave para o domínio!